

Bloco 4.3, Propostas: tela de listagem — status de entrega

Data: 2026-09-04. Branch: feature-dev-breno-automacao. Working tree (sem commit, conforme a regra de só commitar sob pedido).

O que o bloco é

Tirar `/propostas` do `<EmConstrucao />` e entregar a listagem no mesmo padrão de `/orcamentos` e `/fd`: busca, filtros, paginação e estado inteiro na URL. A base veio do bloco 4.2 (tipos, helpers puros e guard de perfil), então aqui não houve regra nova de negócio — o trabalho foi montar a tela sobre o que já existia e resolver a única diferença estrutural de Propostas, que é o cliente não estar na tabela.

Atividade 1: colunas

Concluída. `propostas-table.tsx` sobre o `DataTable` genérico, sete colunas na ordem pedida: Número, Obra, Cliente, Data emissão, Validade, Valor final, Status.

Duas notas:

- **Obra sai como** `codigo_obra – nome`, um campo só, igual ao label do select de obra em `/fd`. Duas colunas separadas gastariam largura numa tabela que já tem sete.
- **Cliente vem de** `p.obra?.cliente?.nome`, dois níveis, porque o JOIN é aninhado (ver atividade 2). Data, validade e valor usam `formatDate` e `formatCurrency` com `tabular-nums`, como nas outras listagens.

Atividade 2: busca por número, obra e cliente

Concluída, e é o ponto que exigiu decisão.

Propostas **não tem** `cliente_id` — o cliente vem por obra (`propostas → obras → clientes`), como o bloco 4.2 já havia registrado em `PropostaListItem`. Consequência prática: não existe um `.or()` que alcance o nome do cliente, porque PostgreSQL não filtra por coluna de tabela aninhada dentro de um `or`. A busca resolve os IDs antes, no mesmo espírito do que `/orcamentos` faz com cliente, com uma diferença:

O passo de obra é feito em memória. A página já carrega a lista completa de obras para alimentar o select do filtro. Reusar essa lista para o match por nome e código da obra economiza uma consulta — sobra só a de `clientes`, que não tem como ser evitada. O helper é puro e testado:

```
obraIdsDaBusca(obras, busca, clienteIds): string[]
```

O `.or()` final da query fica com dois termos, `numero.ilike` e `obra_id.in(...)`, e o segundo é omitido quando nenhuma obra casa — igual ao tratamento de `clienteFilter` em `/orcamentos`.

Feito além do checklist: a busca também casa `codigo_obra`. O checklist pede nome da obra, mas quem digita "OB-2026" espera achar, e é a mesma linha de código.

Correção de robustez: `sanitizeBusca` remove vírgula, parêntese e aspas do termo antes de montar o `.or()`. O `.or()` do PostgreSQL recebe os filtros como uma string separada por vírgula, com os valores de `in(...)` entre parênteses, então um desses caracteres digitado na busca não devolve "nada encontrado" — ele quebra o parser e **derruba a query inteira**. As listagens de orçamentos e FD tinham o mesmo buraco e passaram a usar o helper compartilhado.

Atividade 3: filtros de obra, status e período

Concluída. `filters.tsx` client component, três selects mais o campo de busca com debounce de 300 ms, cada mudança escrevendo na URL e zerando `page` — o mesmo componente de `/fd`, adaptado.

- **Obra:** select alimentado pelo servidor via prop `obraOptions`, valor é o `id`, filtro vira `eq('obra_id', ...)`.
- **Status:** as quatro opções vêm de `STATUS_PROPOSTA_OPTIONS`, do 4.2, em vez de uma lista literal como em orçamentos. A querystring passa por `isPropostaStatus` antes de virar `eq`, então `?status=qualquercoisa` é ignorado em vez de retornar erro do banco.
- **Período de emissão:** 30d, 90d e ano, sobre `data_emissao`.

Vencidas entrou como quinta opção do select de status, resolvendo a especificação da pendência 3 do bloco 4.2. Como vencida **não é status do banco**, ela não pode viajar em `?status=`: a URL usa `?vencidas=1` e a query filtra `status = 'enviada'` com `data_validade < hoje`.

A decisão de UI que vale registro: **vencidas ocupa uma linha do mesmo select de status em vez de um controle próprio**. Um checkbox separado permitiria `?status=aprovada&vencidas=1`, combinação contraditória que sempre retornaria lista vazia. No mesmo select, um valor apaga o outro por construção — o handler escreve um e deleta o outro, e o servidor trata `vencidas` antes de `status`.

Atividade 4: paginação de 20

Concluída. `PAGE_SIZE = 20`, `.range(from, to)` com `count: 'exact'` e o componente `Pagination` já existente, reusado sem alteração. Os cinco filtros seguem em `extraParams`, então navegar entre páginas não perde estado.

Correção posterior (2026-09-05): pedir uma página depois da última mostrava a faixa vermelha de erro do banco, porque o PostgreSQL recusa o range com 416 e não devolve o `count`. Hoje a listagem redireciona pra primeira página preservando os filtros. Valia para as cinco listagens do sistema — detalhe em `docs/tecnicos/correcoes-listagens-e-obras.md`.

Ordenação: `data_emissao` desc, com dois cuidados que as outras listagens não precisam ter. `data_emissao` é **nullable** em propostas, então vai `nullsFirst: false` — rascunho sem data de emissão fica no fim, não no topo. E existe um segundo `order` por `numero` desc como desempate: sem ele, propostas com a mesma data de emissão podem sair em ordem diferente entre duas requisições, e um registro repetiria na página 2 enquanto outro desapareceria.

Atividade 5: badge de status

Concluída, já estava pronta. As quatro chaves de proposta entraram no `StatusBadge` no bloco 4.2, com exatamente as cores pedidas: rascunho cinza, enviada azul, aprovada verde, rejeitada vermelho. Nada a fazer no componente.

O que a listagem acrescentou foi o **selo âmbar "Vencida"** ao lado do badge, quando `isPropostaVencida` é verdadeiro — a outra metade da pendência 3 do 4.2. Ele é um selo separado, não uma cor do `StatusBadge`, justamente porque não é status: uma proposta vencida continua `enviada` no banco.

Atividade 6: EmptyState e botão Nova proposta

Concluída. `canCreate` é admin ou comercial, checado por `getCurrentProfile`, e governa tanto o botão do topo quanto o CTA do `EmptyState`. Ícone `FileSignature`, na linha do `FileText` de orçamentos e do `HandCoins` de FD.

Os dois estados vazios são distintos, como nas outras listagens: sem nenhum registro aparece o `EmptyState` com o CTA; com filtro aplicado e zero resultados aparece a caixa "Nenhuma proposta encontrada com esses filtros", que não oferece criar.

Feito além do checklist

Dois placeholders para a listagem não terminar em 404. A tela tem linha clicável (`rowHref`) e botão de criar, e nenhum dos dois destinos existe ainda — são os blocos seguintes. Ficaram dois `page.tsx` mínimos com `<EmConstrucao />`: `propostas/nova` e `propostas/[id]`. O de criação já leva o **guard de escrita** (`redirect('/propostas')`) para quem não é admin nem comercial), porque o `layout.tsx` de propostas libera `visualizador` também — a rota de criar não pode herdar essa permissão, e deixar o guard para depois é o tipo de coisa que não volta.

A triplicação de `computePeriodoCutoff` foi eliminada. A mesma função estava copiada em quatro lugares (`/orçamentos`, `/fd` e as duas rotas de export em `api/export`), e Propostas seria a quinta. Virou `src/lib/listagem.ts`, com `PERIODO_OPTIONS` junto — as opções do select também estavam duplicadas nos dois `filters.tsx`. Os quatro chamadores passaram a importar; nenhum mudou de comportamento, o parâmetro `hoje` é opcional e só existe para o teste poder fixar a data. `src/lib/listagem.ts` não tem `use client` nem import de React, então serve Server Component, Server Action e Client Component, igual aos outros helpers de `src/lib`.

Verificação

Rodada em 2026-09-04, na entrega deste bloco, com o complemento de 2026-09-05 descrito abaixo. Os números crescem a cada bloco; o estado atual sai de `npm run validar`.

COMANDO	RESULTADO
<code>npm test</code>	31 casos, todos passando (eram 17; 46 em 2026-09-05)
<code>npx tsc --noEmit</code>	limpo
<code>npm run lint</code>	sem warnings nem erros
<code>npm run build</code>	compila, 29 rotas (31 em 2026-09-05)

Os 14 casos novos cobrem o que a listagem tem de lógica pura: os seis de `obraIdsDaBusca` (nome, código, união com os clientes que casaram sem duplicar, busca vazia e sem match) e os oito de `listagem.ts` (os três cutoffs, o valor desconhecido da querystring virando "sem filtro", a garantia de não mutar a data recebida, e o `sanitizeBusca` removendo o que quebra o `.or()` sem estragar acento, hífen e barra).

Seguindo a lição do 4.1, o retorno do `select()` usa `as` e não `as unknown as`, para o `tsc` continuar comparando a string da query com o `Database` gerado — se uma coluna do select deixar de existir, o build acusa.

Verificado depois, contra gc-dev. O bloco fechou primeiro com a pendência "nada rodou contra o banco". Ela foi resolvida junto com o plano de validação (`docs/tecnicos/plano-validacao.md`), e o resultado real é:

CAMADA	RESULTADO
runtime (<code>next start</code> , sessão real)	9/9 rotas: <code>/propostas</code> devolve 200 com <code><table></code> e as colunas, sem sessão redireciona pro <code>/login</code>
dados (queries sob RLS)	6/6 checagens. O JOIN aninhado <code>obra:obras(codigo_obra, nome, cliente:clientes(nome))</code> volta objeto, não array, com 1 proposta real em gc-dev

O filtro `?vencidas=1` responde 200 e a query é aceita, mas volta **0 linha**: a única proposta de gc-dev tem `data_validade` nula. A lógica de vencida está validada por teste unitário e a query pelo banco, mas o caminho com dado positivo ainda não foi exercitado — precisa de uma proposta `enviada` com validade no passado no seed.

Correção de número: este documento dizia "22 rotas (eram 19)". São 29 (eram 27) — o 22 saiu de ler `Generating static pages (23/23)` em vez da tabela de rotas. A camada build agora compara a lista contra `scripts/rotas-esperadas.txt`, então esse tipo de erro passou a ser detectado em vez de contado à mão.

O que vem depois

O roteiro do 4.2 segue valendo, com o item 1 agora fechado.

Atualização de 2026-09-05: os sete itens foram entregues nos blocos 4.4 (criação e edição), 4.5 (detalhe, anexos, exportação e painel financeiro) e 4.6 (mudança de status). A lista fica como registro do que estava planejado quando este bloco fechou.

1. ~~Listagem~~. **Este bloco.**
2. **Criação e edição.** `proposta-form-helpers.ts` com schema zod e o par `formValuesToPayload / propostaToFormValues`, mais `proposta-form.tsx`. O form trabalha em 0 a 100 e o payload em fração (`pctToFraction / fractionToPct`), o schema reproduz `desconto <= valor_total` e chama `validarSomaPct`. Substitui o placeholder de `propostas/nova`.
3. **Detalhe e mudança de status.** `[id]/page.tsx`, `detail-header.tsx` (com o mesmo selo "Vencida" da listagem) e `status-dialog.tsx` sobre `canChangePropostaStatus`. Substitui o placeholder de `propostas/[id]`.
4. **Anexos**, sobre a coluna `anexos` jsonb e o bucket `anexos`.
5. **Permissão de escrita nas Server Actions** — o guard de layout protege a rota, não a ação.
6. **Exportação.** `api/export/propostas` mais o `ExportButton` na listagem. Ficou de fora **deste bloco** de propósito: o checklist do 4.3 não pede, e o botão sem endpoint só dá erro. Entrou em 2026-09-05, junto do 4.5 — quem abrir `/propostas` hoje vê o botão. Quando entrar, a rota já tem `computePeriodoCutoff` e `sanitizeBusca` compartilhados, e precisa repetir a resolução de IDs de `obraIdsDaBusca` para o filtro de busca bater com o da tela.
7. **Painel financeiro da proposta**, se entrar no escopo (view `propostas_financeiro`).

Pendências

1. ~~Listagem não verificada contra o banco.~~ **Resolvida.** As camadas runtime e dados de `scripts/validar.sh` rodaram contra gc-dev: 9/9 rotas e 6/6 queries. Detalhe na seção de verificação.
2. ~~Filtro de vencidas sem dado positivo em gc-dev.~~ **Resolvida em 2026-09-05** por `scripts/seed-propostas-dev.mjs`: cria (ou atualiza) `SEED-VENCIDA-001`, proposta enviada com validade 15 dias no passado. É idempotente pelo `numero`, que é unique por empresa, então rodar de novo não acumula linha. Roda autenticado como usuário real, o que de quebra exercitou a policy `"Propostas: comercial insere"` — escrita no banco é a parte declarada como manual no plano de validação. Depois dele, `?vencidas=1` deixou de ser "query válida, mas não exercitada": a tela renderiza a linha com o selo âmbar, e a checagem de dados passou a **exigir** pelo menos uma linha, falhando se o seed sumir.
3. ~~Herança do 4.1: supabase-migration-repair~~ : **Resolvida para gc-dev em 2026-09-05.** O `SUPABASE_ACCESS_TOKEN` do `.env.local` funcionou:

```
supabase link --project-ref gzbmhgnpoehormnidmgg
supabase migration list      # 10 locais, 10 remotos VAZIOS
supabase migration repair --status applied <as 10 versões>
supabase migration list      # 10 locais = 10 remotos
```

O `repair` grava só metadados em `supabase_migrations.schema_migrations`, não toca schema nem dado, e é reversível com `--status reverted`. Com o histórico alinhado, um `db push` deixou de tentar reaplicar a `revisao_schema` (ex-004), que é destrutiva e não idempotente — foi o que permitiu, no bloco 4.6, subir a migration do histórico com segurança.

Seguem abertas as duas partes que não dependiam disso: revogar o token antigo (só pelo painel) e repetir `link` + `repair` em gc-prod quando despausar.

O que, dentro deste bloco, seguia sem exercício

Três caminhos da listagem tinham sido entregues sem nunca rodar. Dois fecharam em 2026-09-05, e o esforço de fechá-los **encontrou dois bugs reais** — está em `docs/tecnicos/correcoes-listagens-e-obras.md`.

1. **O EmptyState de "nenhuma proposta cadastrada". Segue aberto.** gc-dev tem duas propostas, e o ramo só renderiza com zero — o que exigiria uma segunda empresa só pra isso. O que a validação cobre é o outro estado vazio, o de filtro sem resultado (`?busca=zzz-inexistente`). O que fechou no lugar foi o bug que *mascarava* o assunto: `?page=99` não mostrava estado vazio nenhum, mostrava a caixa vermelha de erro do banco.
2. ~~A paginação com mais de 20 registros.~~ **Resolvida.** `/orcamentos?page=2` (28 registros em gc-dev) entrou na validação e exercita o `Pagination` de verdade — Anterior, Próxima e tabela. É o mesmo componente que Propostas usa, sem alteração. E as cinco listagens ganharam asserção de página fora do alcance, que é o caminho que estava quebrado.
3. ~~Os filtros vistos por outro perfil.~~ **Resolvida.** A senha do `visualizador@teste.com` foi redefinida (`scripts/resetar-senha-dev.mjs`) e a camada runtime passou a fazer login com um segundo usuário. Quatro asserções: a listagem sem o botão "Nova proposta", `/propostas/nova` redirecionando, o detalhe sem "Mudar status" nem "Excluir", e a edição redirecionando.

Estado do repositório

As mudanças ficaram **no working tree**, conforme a regra de trabalho, e foram **commitadas por você** em 2026-09-05, no commit `0c1521b` ("task 4.3 concluídas"), que levou junto os três documentos deste bloco. As correções de número e de pendência feitas depois (esta revisão inclusive) estão no working tree em cima dele.

O push para `cremalet/gc-sistema` (remote `upstream`) **funciona**: a branch foi enviada para lá em 2026-09-05. **Correção dupla deste documento**: ele dizia primeiro que os commits existiam "só no clone local" e depois que o push era bloqueado por falta de permissão em `cremalet`. As duas coisas eram

falsas — a primeira porque o fork `origin` (`brenocmlo/gc-sistema`) já os tinha, e a segunda porque `brenocmlo` é **colaborador** do repositório da equipe. A afirmação de bloqueio nasceu no documento do 4.1 e foi repetida daqui em diante sem nunca ter sido testada; bastava um `git push --dry-run`.

Sobre o que ficou de fora, de propósito: `package-lock.json` não existe mais nem é rastreado (o projeto usa pnpm), os `.html` de `docs/` foram removidos porque só `.md` e `.pdf` ficam versionados, e `supabase/criar_usuario_admin.sql` entrou no commit `0c1521b` — o untracked que sobrou é `supabase/criar_usuario_admin_leticia.sql`, criado por você.

Arquivos

ARQUIVO	O QUE É
<code>src/app/(app)/propostas/page.tsx</code>	reescrito. A listagem, sobre o select do 4.2
<code>src/app/(app)/propostas/filters.tsx</code>	novo. Busca, obra, status + vencidas, período
<code>src/app/(app)/propostas/propostas-table.tsx</code>	novo. As sete colunas e o selo "Vencida"
<code>src/app/(app)/propostas/nova/page.tsx</code>	novo. Placeholder com guard de escrita
<code>src/app/(app)/propostas/[id]/page.tsx</code>	novo. Placeholder do detalhe
<code>src/lib/listagem.ts</code>	novo. <code>computePeriodoCutoff</code> , <code>PERIODO_OPTIONS</code> , <code>sanitizeBusca</code>
<code>src/lib/listagem.test.ts</code>	novo. 8 casos
<code>src/lib/propostas.ts</code>	<code>obraIdsDaBusca</code> e <code>ObraBuscaRef</code>
<code>src/lib/propostas.test.ts</code>	6 casos novos, sobre a busca
<code>src/app/(app)/orcamentos/page.tsx</code> , <code>filters.tsx</code>	passam a usar <code>@/lib/listagem</code>
<code>src/app/(app)/fd/page.tsx</code> , <code>filters.tsx</code>	passam a usar <code>@/lib/listagem</code>
<code>src/app/api/export/{fd,orcamentos}/route.ts</code>	passam a usar <code>@/lib/listagem</code>
<code>src/components/StatusBadge.tsx</code>	inalterado, já tinha as cores pedidas
<code>docs/4.3-status-entrega.md</code>	este documento